

蓝图基础知识

一、定义

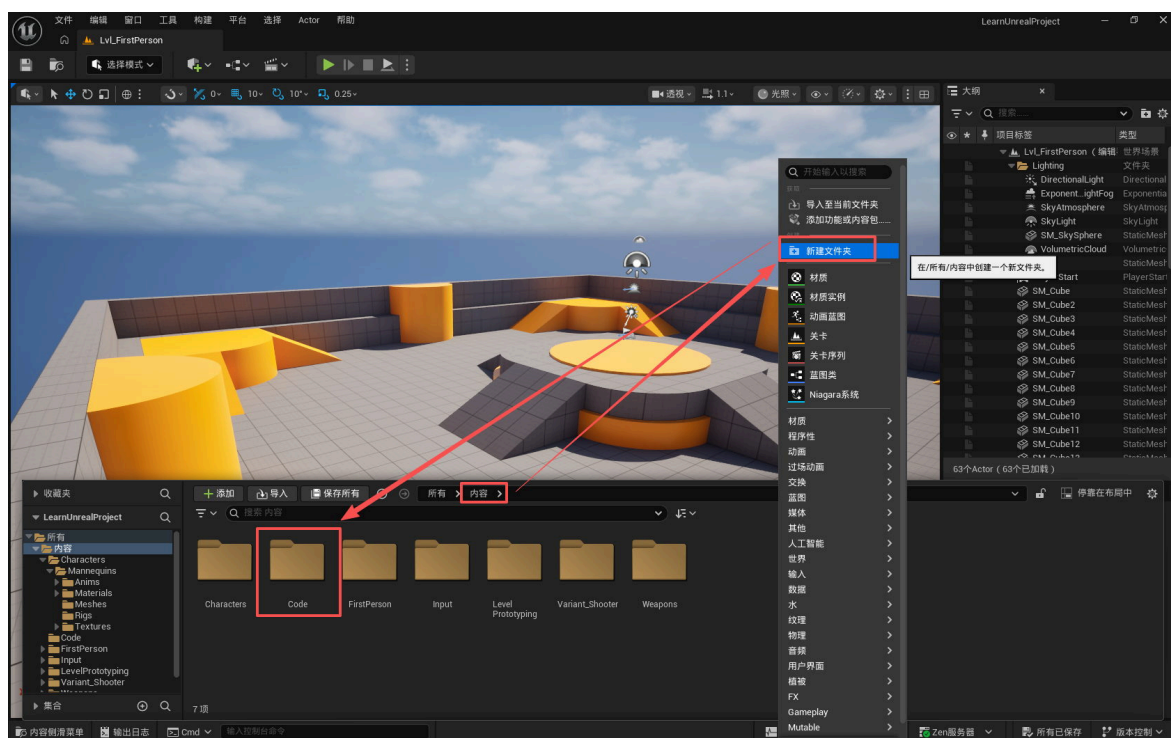
蓝图是一种可视脚本，可用于创建逻辑，它称为可视化脚本，**蓝图视觉效果脚本（Blueprint Visual Scripting）**系统是完善的gameplay脚本系统，在虚幻编辑器中使用基于节点的界面来创建gameplay元素。就像许多常用的脚本编写语言，它可以用于在引擎中定义以object为导向的类或object。在使用虚幻引擎时，经常会发现使用蓝图定义的object被统称为"蓝图"。

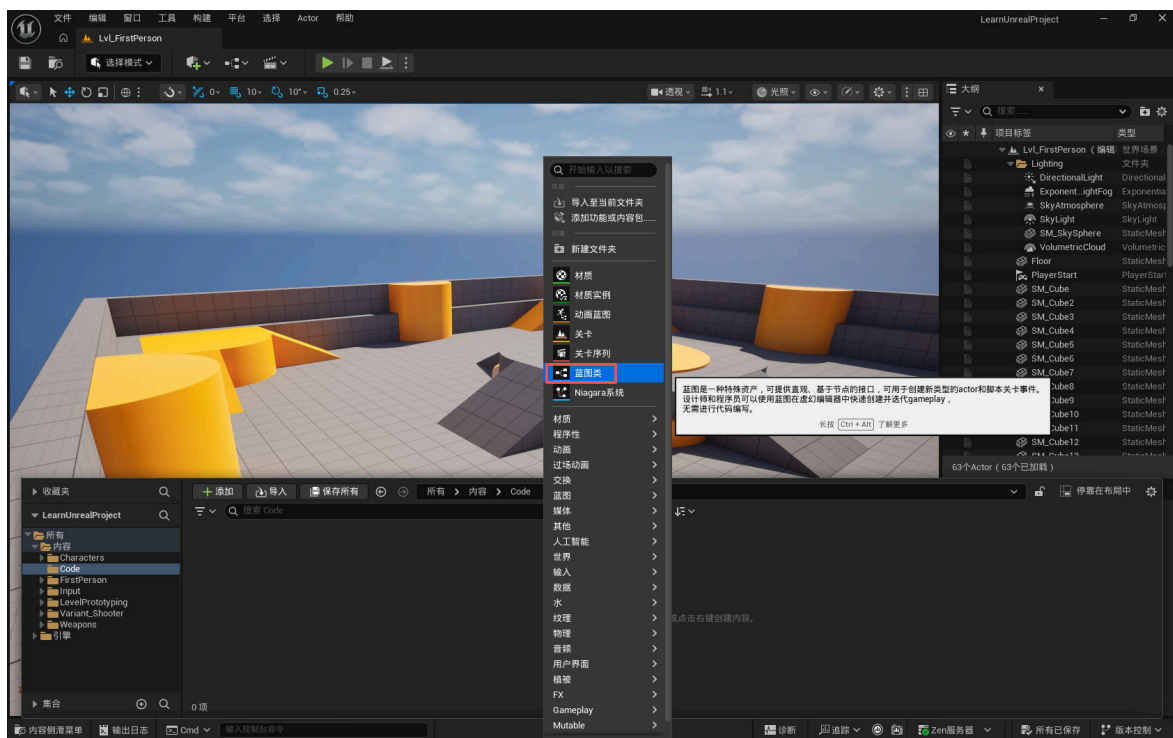
使用我们具有特点功能的节点来创建所有逻辑，这样，就无需使用C++代码即可创建逻辑，**优点**是无需编写代码即可快速创建逻辑，在不了解C++或者不是程序员的情况下创建项目，**缺点**是无非访问C++中的所有功能，除此之外，与C++相比，蓝图的性能较慢，如果打算制作一个小项目，那你完成可以使用蓝图，甚至独立游戏，完全可以使用蓝图来进行制作。

二、创建蓝图

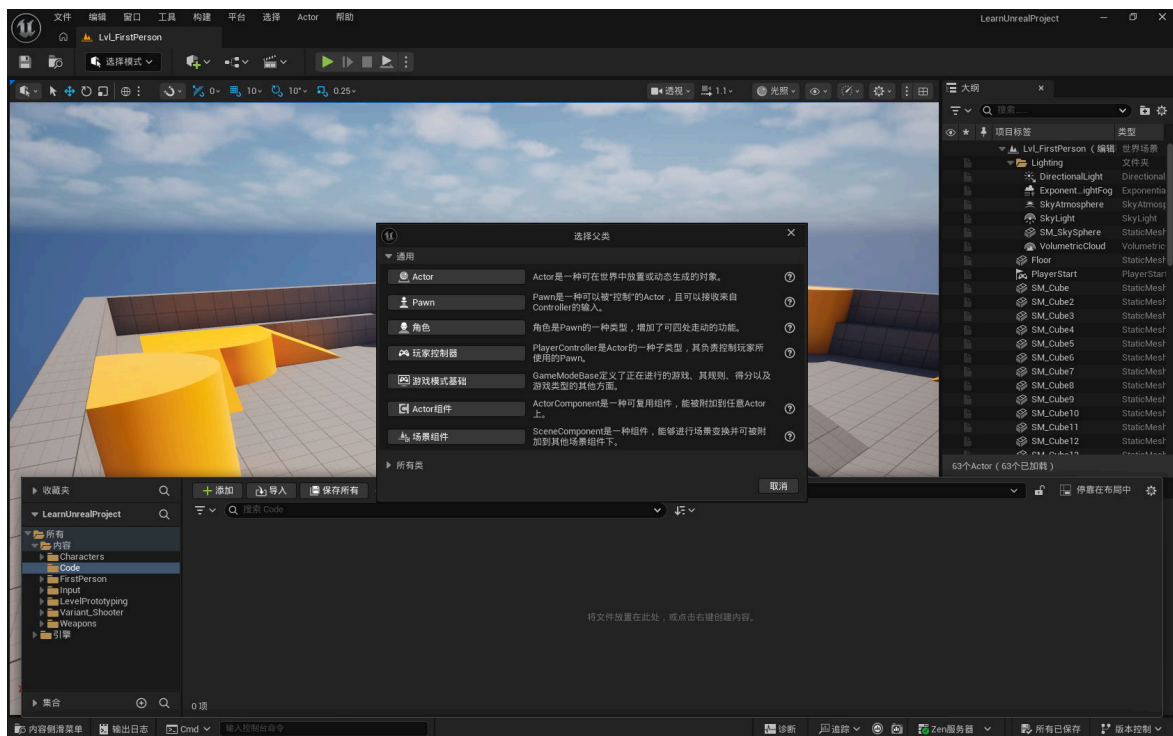
1. 步骤

1. 在内容侧滑菜单的内容目录下右键新增文件夹，重命名为Code，存放后续蓝图
2. 在新建的Code目录下选择右键菜单中的蓝图类，选择需要的父类





2. 各父类讲解



2.1 Actor

定义：**Actor** 是可以放置在关卡中的任意对象，如摄像机、静态网格体或玩家出生点。**Actor**支持3D变换，如平移、旋转和缩放。你可以通过gameplay代码（C++或蓝图）来创建和销毁**Actor**。在C++中，**AActor** 是所有**Actor**的基类。

2.2 Pawn

定义：**Pawn**是可那些由玩家或 AI 控制的所有 Actor 的基类。**Pawn** 是玩家或 AI 实体在游戏场景中的具化体现。

这说明，**Pawn** 不仅决定了玩家或 AI 实体的外观效果，还决定了它们如何与场景进行碰撞以及其他物理交互。某些游戏可能在游戏没有可见的玩家模型或替身（Avatar），因此这点在某些情况下可能会令人困惑。不过，无论如何，**Pawn** 仍代表着玩家或实体在游戏中的物理方位、旋转角度等。

角色是一种特殊的、可以行走的 **Pawn**。默认情况下，**控制器（Controllers）** 和 **Pawn** 之间是一对一的关系；也就是说，每个控制器在某个时间点只能控制一个 **Pawn**。此外，在游戏期间生成的 **Pawn** 不会被控制器自动控制。

2.3 角色

定义：添加 **CharacterMovementComponent**、**CapsuleComponent** 和 **SkeletalMeshComponent** 后，**Pawn** 类可延展为功能完善的**角色类**。角色用于代表垂直站立的玩家，可以在场景中行走、跑动、跳跃、飞行和游泳。此类也包含基础网络连接和输入模型的实现。

2.4 玩家控制器

- 定义：**PlayerController**（玩家控制器）是**Pawn**和控制它的人类玩家间的接口。**PlayerController**本质上代表了人类玩家的意愿。当您设置 **PlayerController**时，您需要考虑的一个事情就是您想在**PlayerController**中包含哪些功能及内容。
- 您可以在 **Pawn** 中处理所有输入，尤其是不太复杂的情况下。但是，如果您的需求非常复杂，比如在一个游戏客户端上的多玩家、或实时地动态修改角色的功能，那么最好在 **PlayerController** 中处理输入。在这种情况下，**PlayerController** 决定要干什么，然后将命令（比如"开始蹲伏"、"跳跃"）发布给**Pawn**。
- 注意：某些情况下必须把输入处理或其他功能放到**PlayerController**中。**PlayerController**在整个游戏过程中都是一直存在的，但是**Pawn**可能是临时存在的。
- 举例：在死亡竞技模式的游戏，您可能死了又重生，所以您将获得一个新的 **Pawn**，但是您的**PlayerController**都是一样的。在这个示例中，如果您将分数保存到您的**Pawn**上，那么分数将会重置，但是如果您将分数保存到 **PlayerController**上，它将不会重置。

2.5 游戏模式基础

GameMode（游戏模块）是编译成 DLL 的游戏项目的游戏性相关类的集合。**GameModeBase**定义了正在进行的游戏、其规则、得分以及游戏类型的其他方面。正如引擎本身由一组模块构成一样，每个游戏也是由一个或多个游戏性模块构成的。这些模块类似于引擎以前的版本中的包的概念，它们都是一组相关类的容

器。在虚幻引擎 4 中，由于游戏逻辑都可以通过 C++ 实现，所以模块实际上是 DLL 文件，而不再是包文件。

2.6 Actor组件

Actor 组件可以扩展Actor的行为。Actor组件是一类特殊的对象，可以作为子对象附加到Actor上。Actor组件默认不会复制，但你可以配置任意Actor组件，使其作为其所属Actor的一部分进行复制。Actor组件可以复制自身属性和子对象，也可以像Actor那样调用由Actor组件类定义的远程过程调用（RPC）。

要将Actor组件作为Actor的一部分复制，必须确保：

- 将拥有该Actor组件的Actor设置为可复制。
- 将Actor组件设置为可复制。

Actor组件类型

- **静态Actor组件** 在其所属Actor生成时生成。静态组件在Actor的C++构造函数中创建的默认子对象，或是在蓝图编辑器的组件模式中创建。
- **动态Actor组件** 是在运行时在服务器上生成的Actor组件。动态Actor组件的创建或删除都会被复制到相连的客户端上。动态Actor组件的工作方式类似Actor。

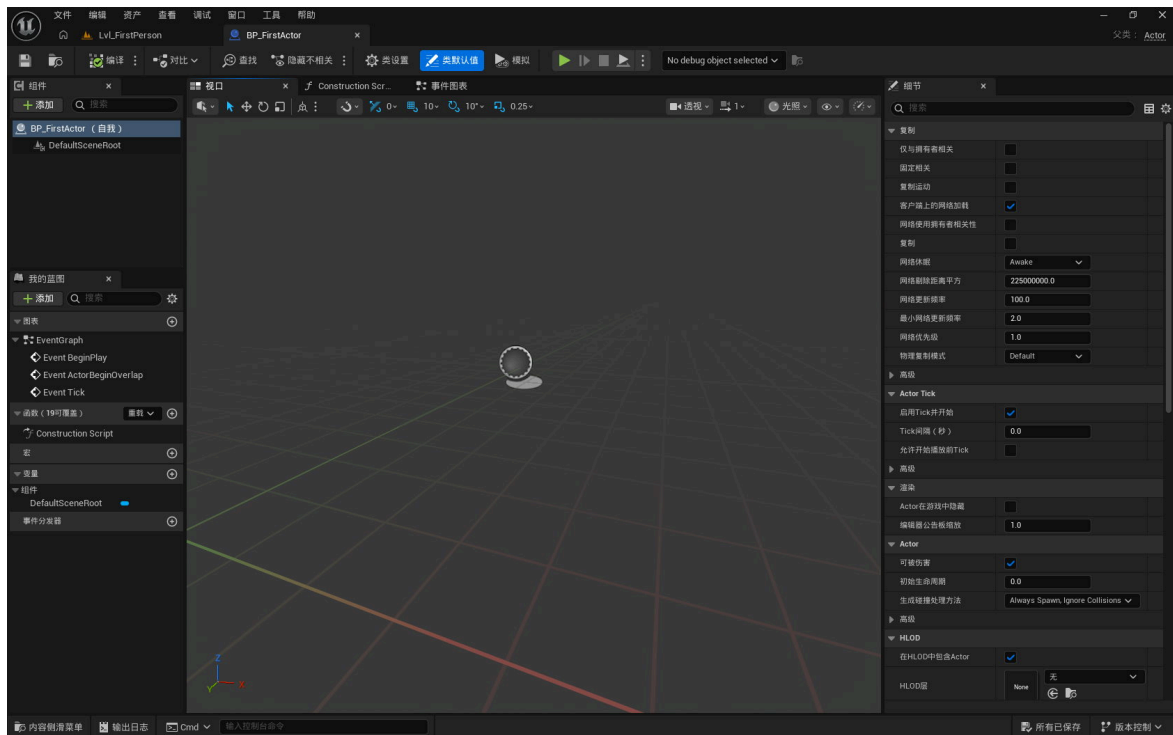
2.7 场景组件

SceneComponent 是一种组件，能够进行场景变换并可被附加到其他场景组件下。

三、Actor界面

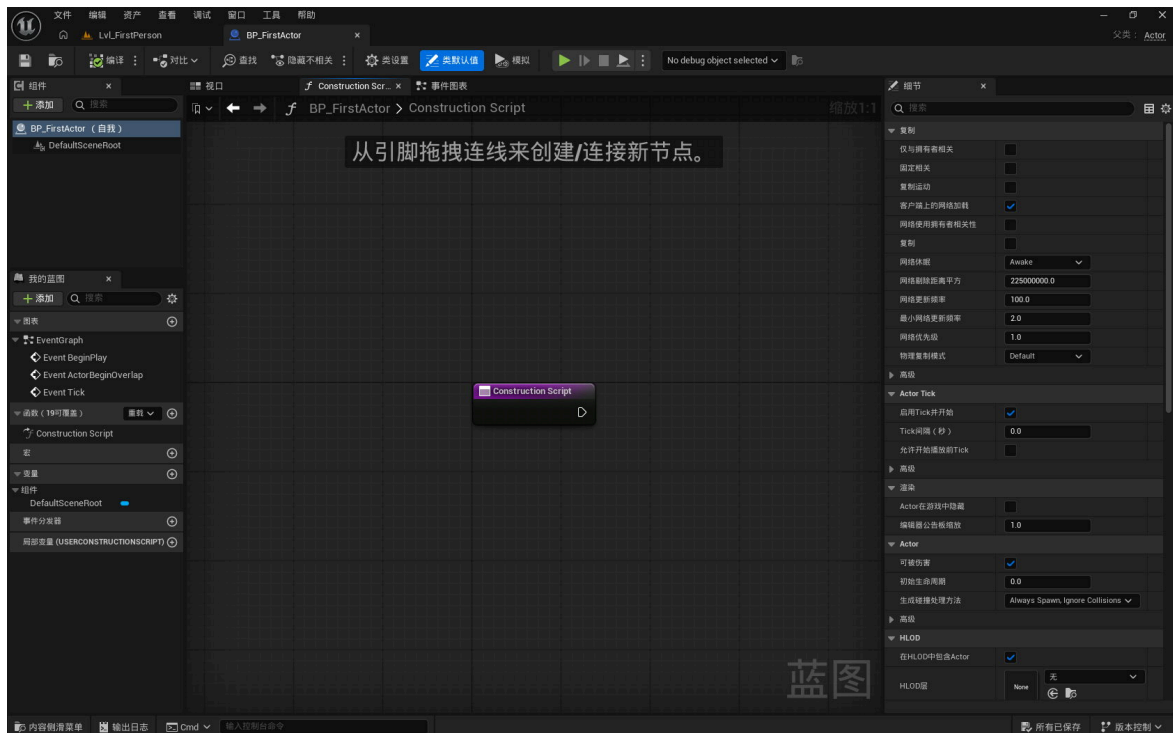
1. 视口

- 组件：是一种可以被添加到Actor的功能性部件。在向Actor添加组件后，该ActorIU可以使用该组件提供的功能，如：
 - 聚光源组件（Spot Light Component）可以让你的Actor发出聚光灯一样的光线。
 - 旋转移动组件（Rotating Movement Component）可以让你的Actor旋转。
 - 音频组件（Audio Component）可以让你的Actor播放音效。
 - 组件必须附加到Actor，不能独立存在。
- 图表：视口中的图表一般用于分类
- 函数：类比于C++或C#中的函数或方法
- 变量：类比于C++或C#中的变量



2. 构造脚本

蓝图类拖拽到场景内之后，构造脚本立即执行；事件图表在点击运行后开始执行



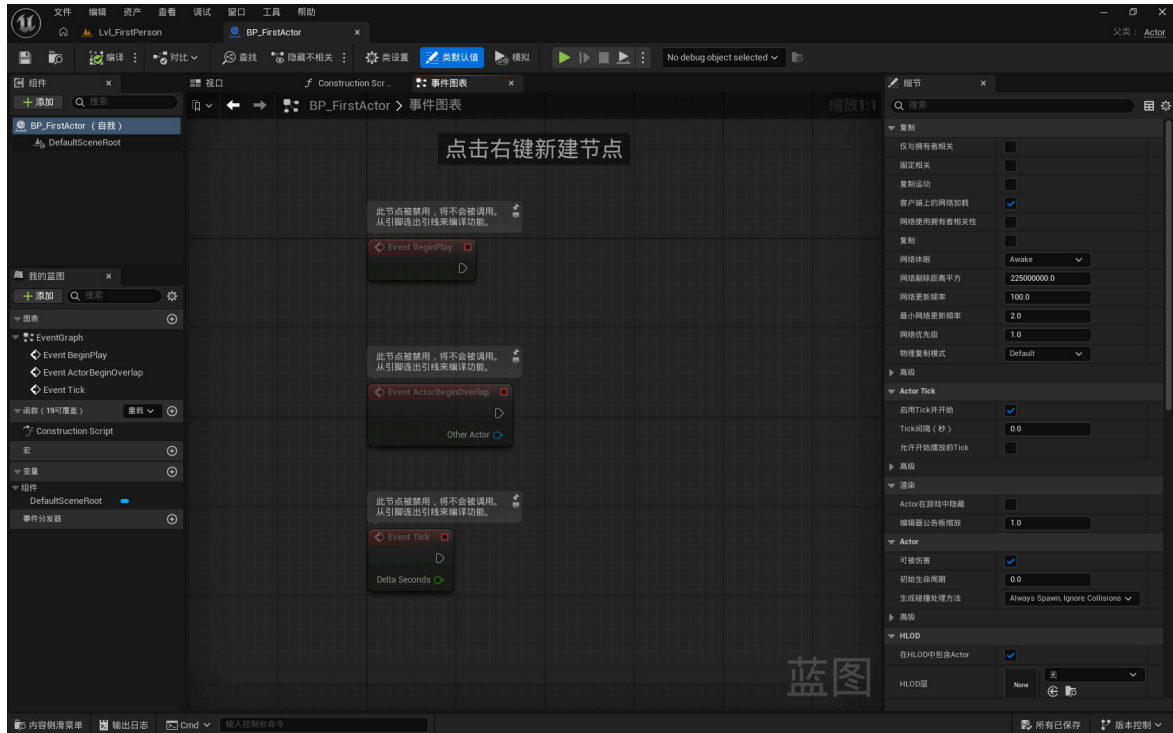
3. 事件图表

3.1 定义

事件图表（Event Graphs）是蓝图图表的最常见类型。每个新建的蓝图类（Blueprint Class）在创建时都将包含一个事件图表。事件图表通常包含蓝图的游戏进程行为的网络，而**事件**、**自定义事件**和**输入**节点则通过事件图表来启动执行流。

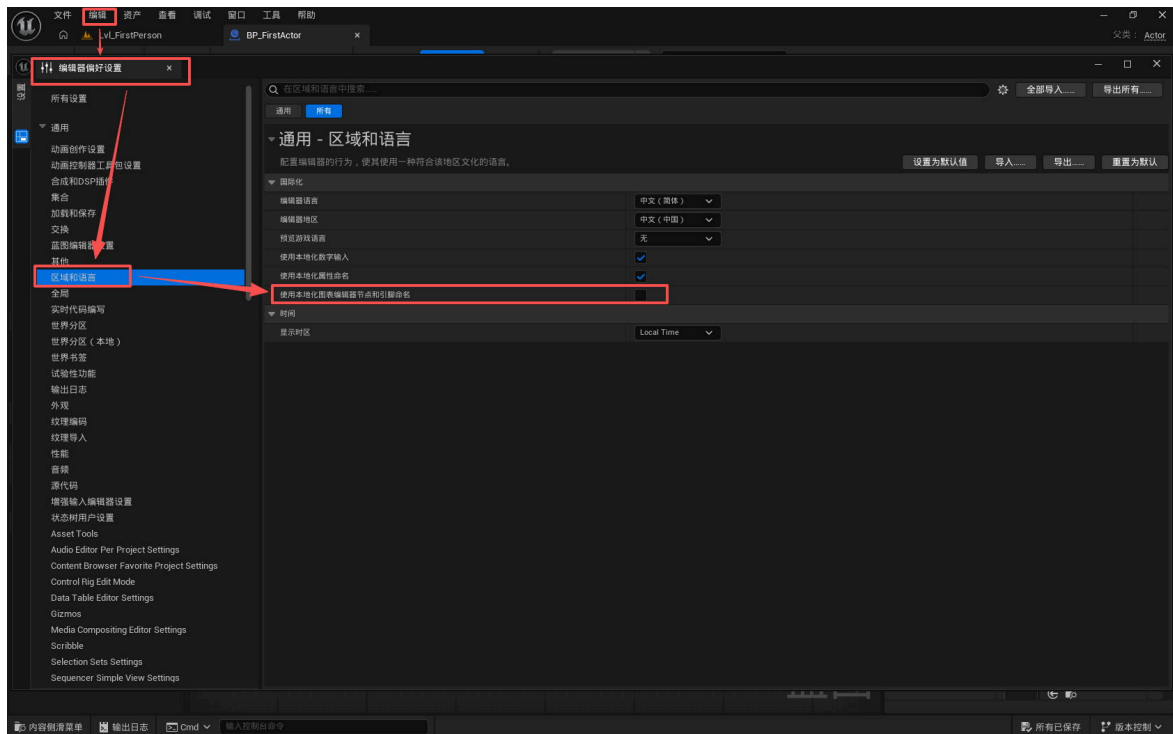
- 事件开始运行：当关卡开始运行时，此节点后续节点会开始运行

- 事件Actor开始重叠：当物体和Actor物体重叠时，此节点后续节点会开始运行（触发器）
- 事件Tick：当Actor生成后，每一帧都会执行此节点后续节点



3.2 节点和引脚的中文化

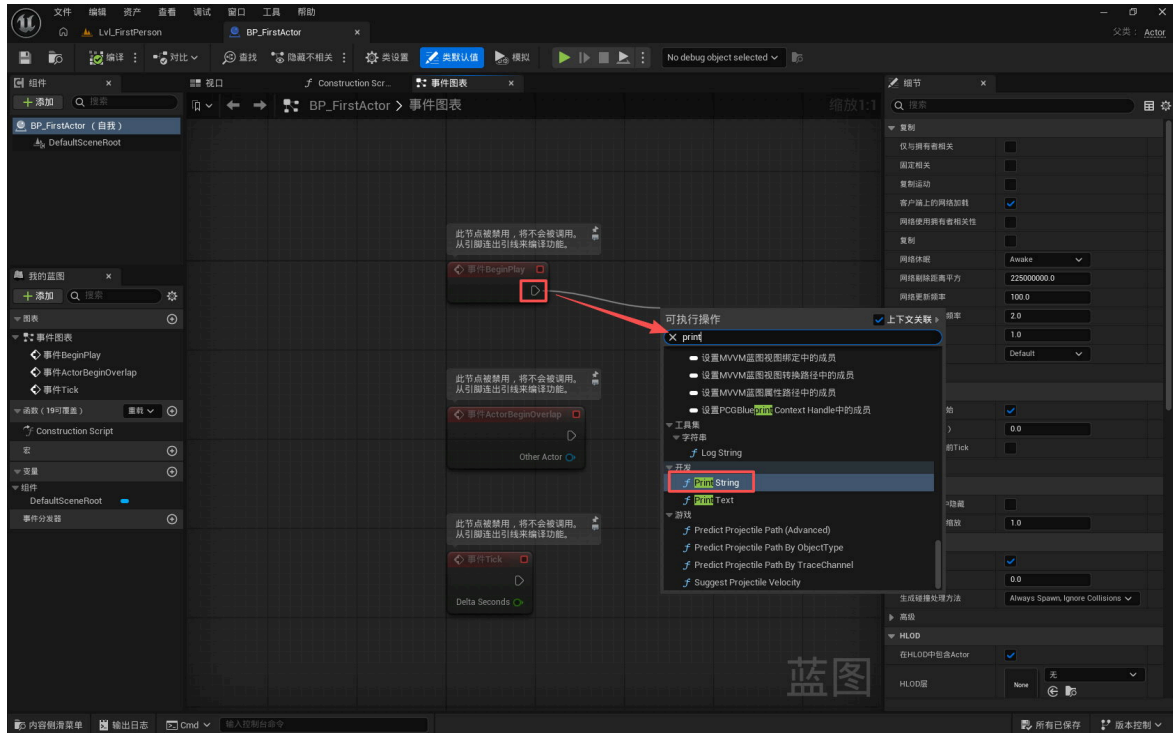
如果事件图表的节点显示为英文，可以选择编辑中的**编辑器偏好设置**，勾选区域和语言中的**使用本地化图标编辑器节点和引脚命名**



4. 第一段代码

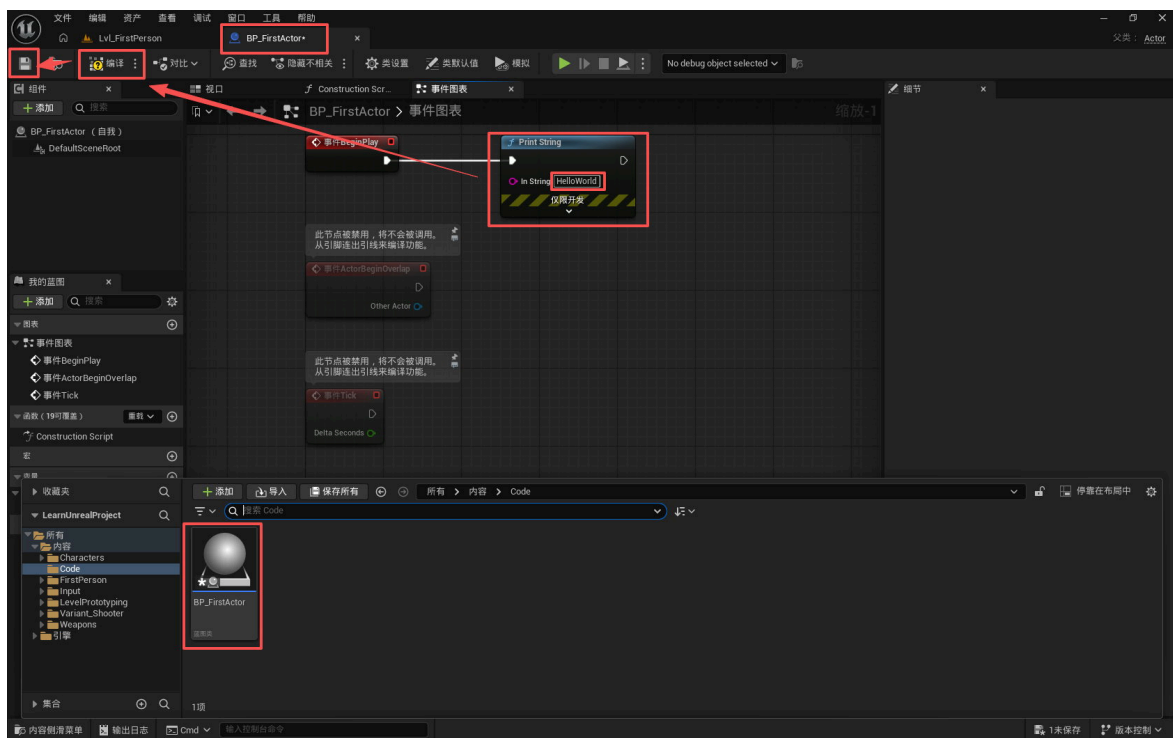
4.1 定义

- 在Actor事件图表界面，左键长按事件开始运行节点右下角的**执行**，向其他位置拖出
- 松开鼠标左键后，弹出如图所示界面，在搜索框中输入**print**，寻找输出函数



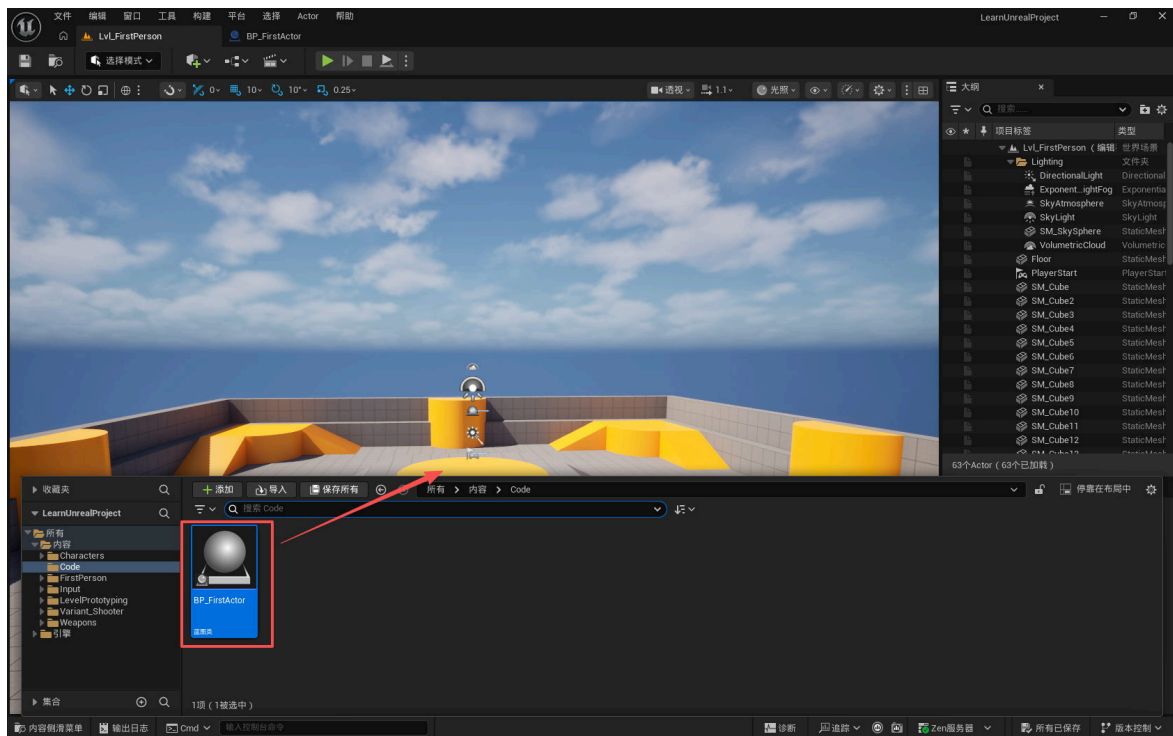
4.2 编写

- 选中输出String的函数后，在**Print String**界面修改默认的**Hello**文本，改为**HelloWorld**文本
- 修改后如图，Actor窗口和内容侧滑菜单关于此Actor物体多出星号标记，表示未保存
- 在运行和保存之前，需要先点击左上角的**编译**，之后点击保存或Ctrl S，界面内星号标记消失
- 仅限开发**是指此节点仅在开发过程中可见，在游戏打包后，外部人员无法获取、观察到此节点效果

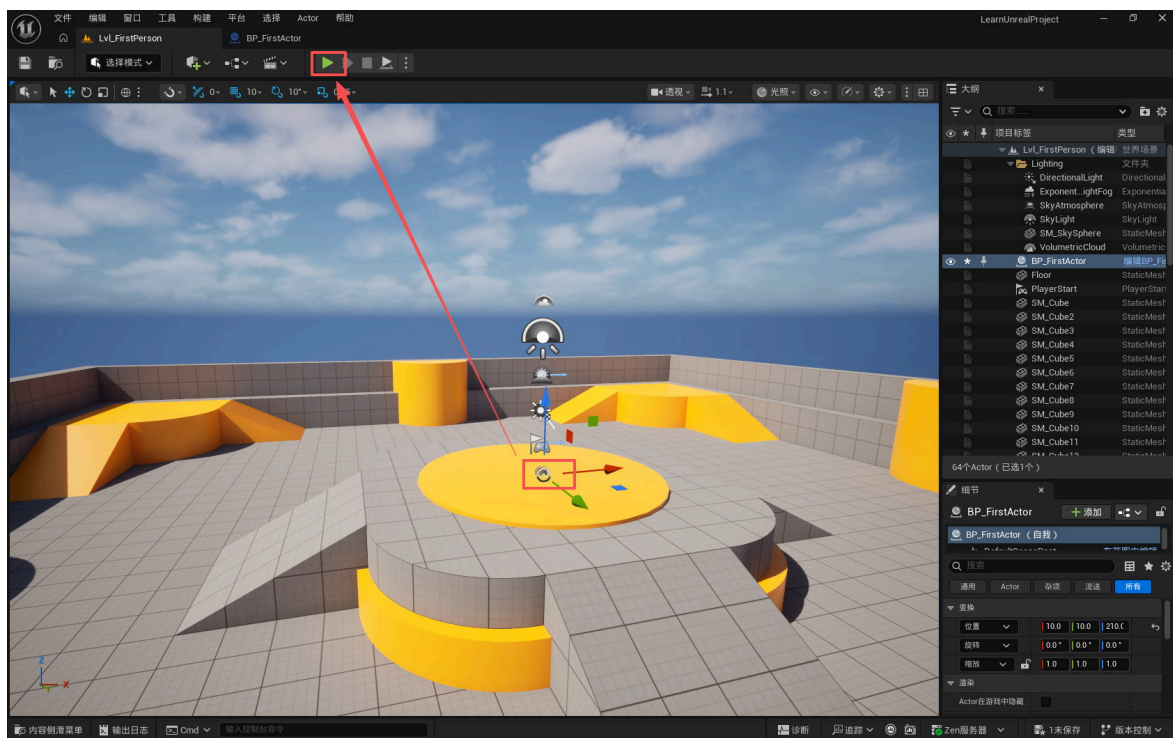


4.3 运行

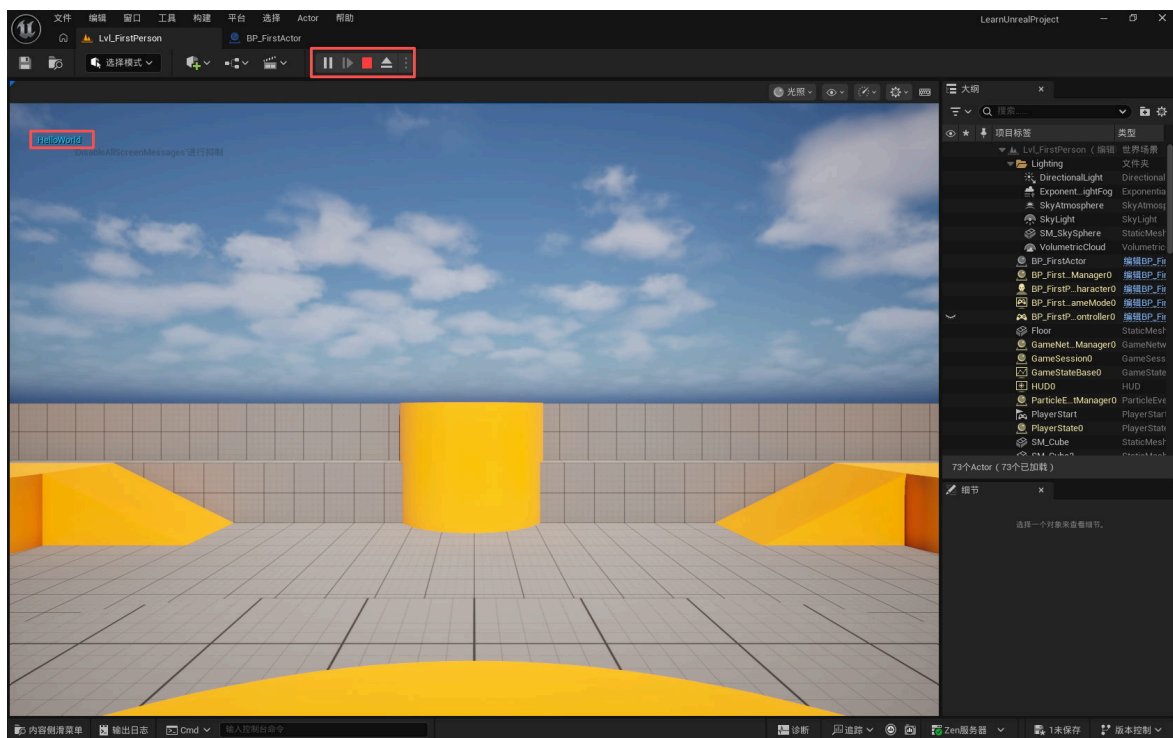
- 事件图表中将代码写好，点击**编译**和保存后
- 返回关卡界面，呼出内容侧滑菜单，长按左键，将**Actor**物体拖拽到关卡内



拖拽到关卡内效果如图，点击开始运行关卡或Ctrl P

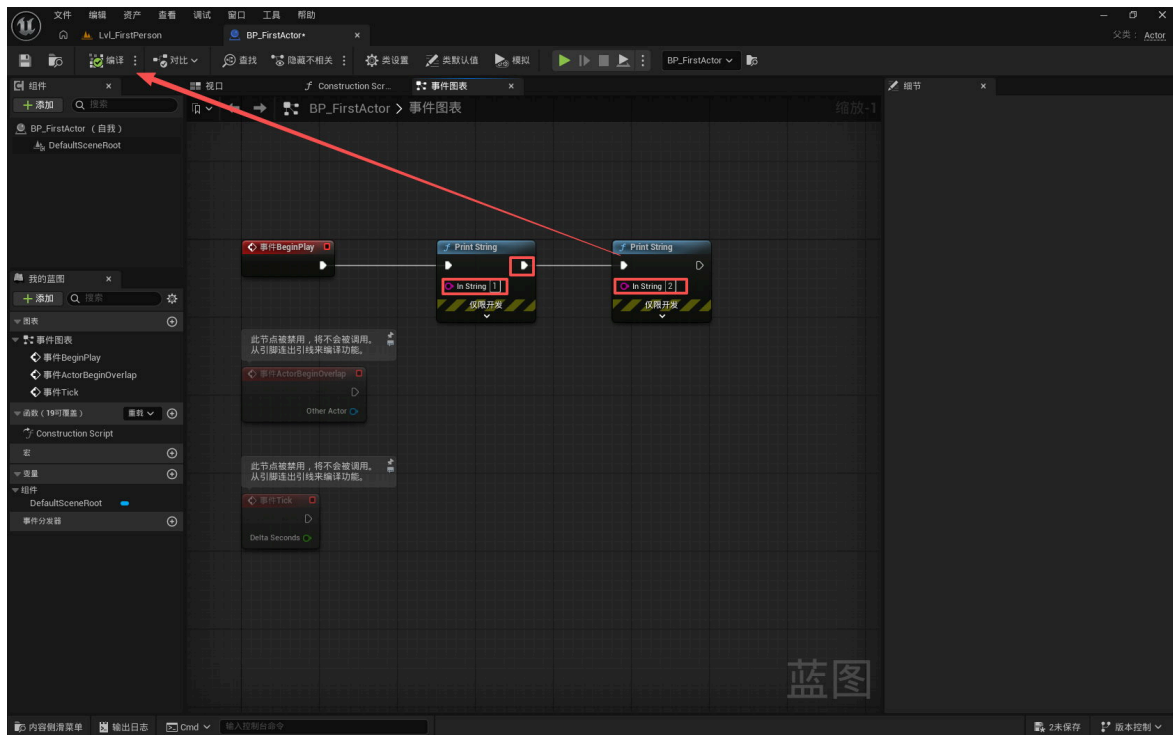


运行效果如图，代表运行成功，可按**esc**退出



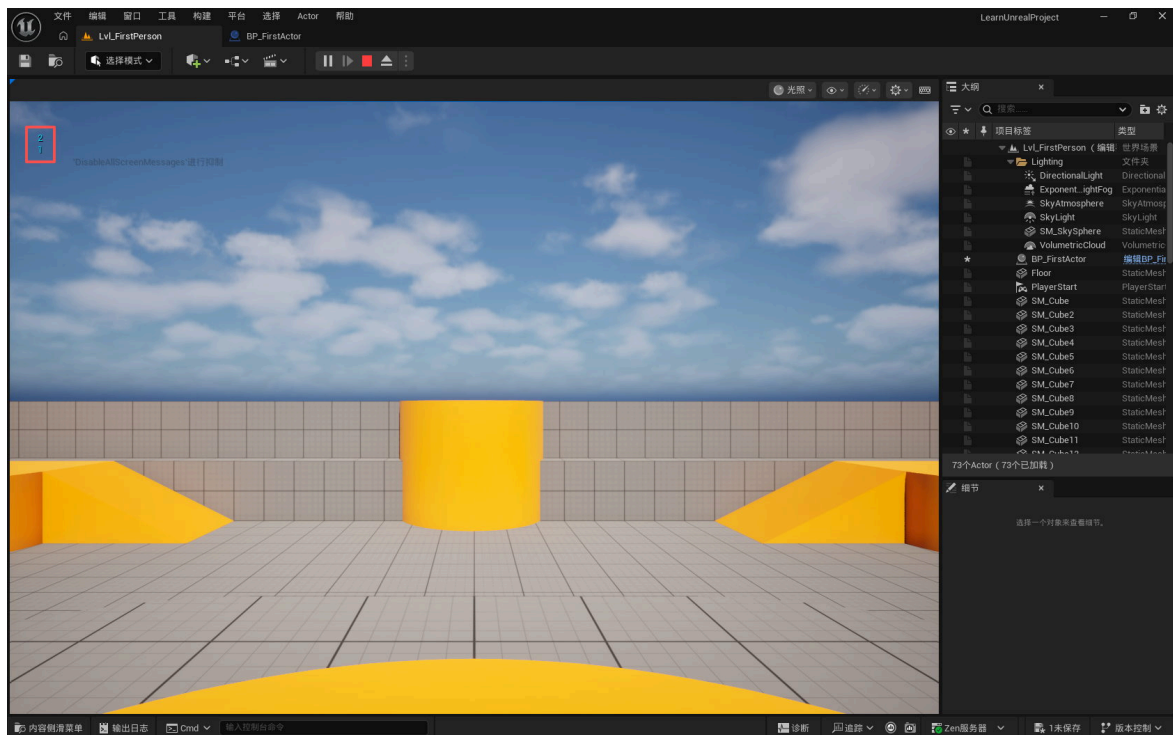
4.4 执行引脚和运行顺序

- 每个节点都有如图的执行引脚，程序会按照前后顺序执行各个节点的代码内容
- 修改原有节点的输出内容，更改为**1**，并在原节点处执行引脚新增新节点，输出内容设置为**2**
- 修改完成后点击编译，保存可以暂时不点，未编译代码无法在关卡内正常运行



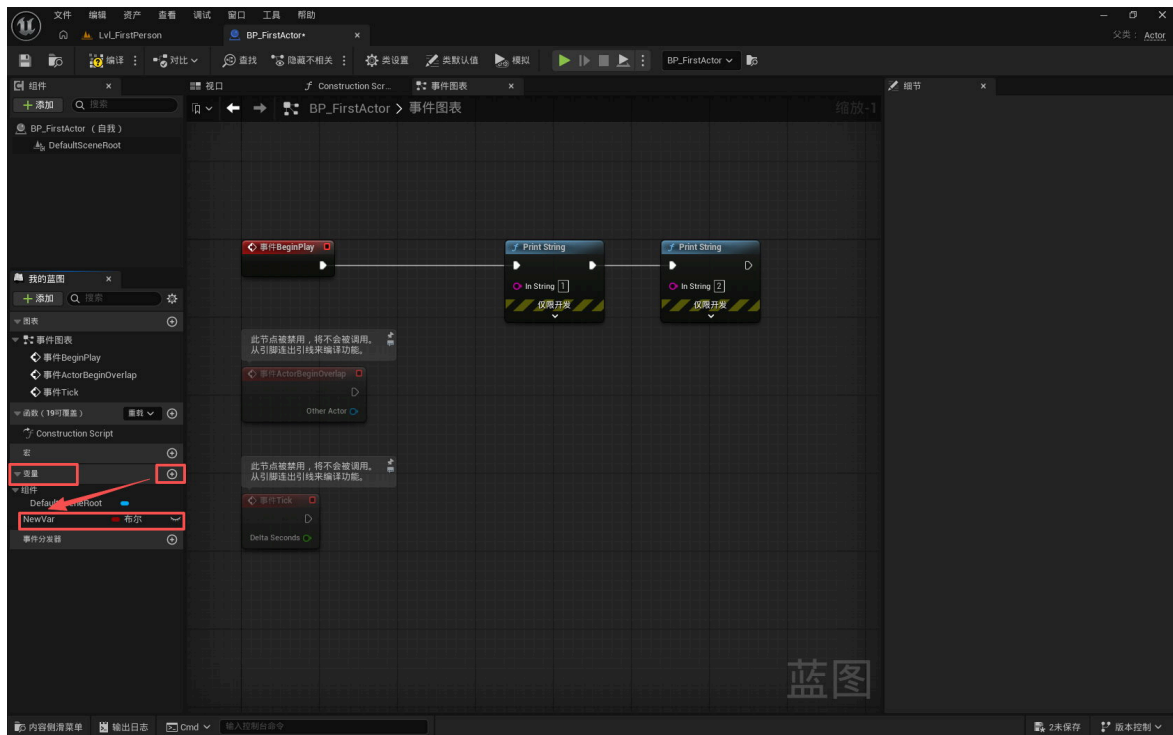
运行效果如图所示，输出的2在上，1在下

顺序为出栈顺序，先执行的在下方，后执行的在上方

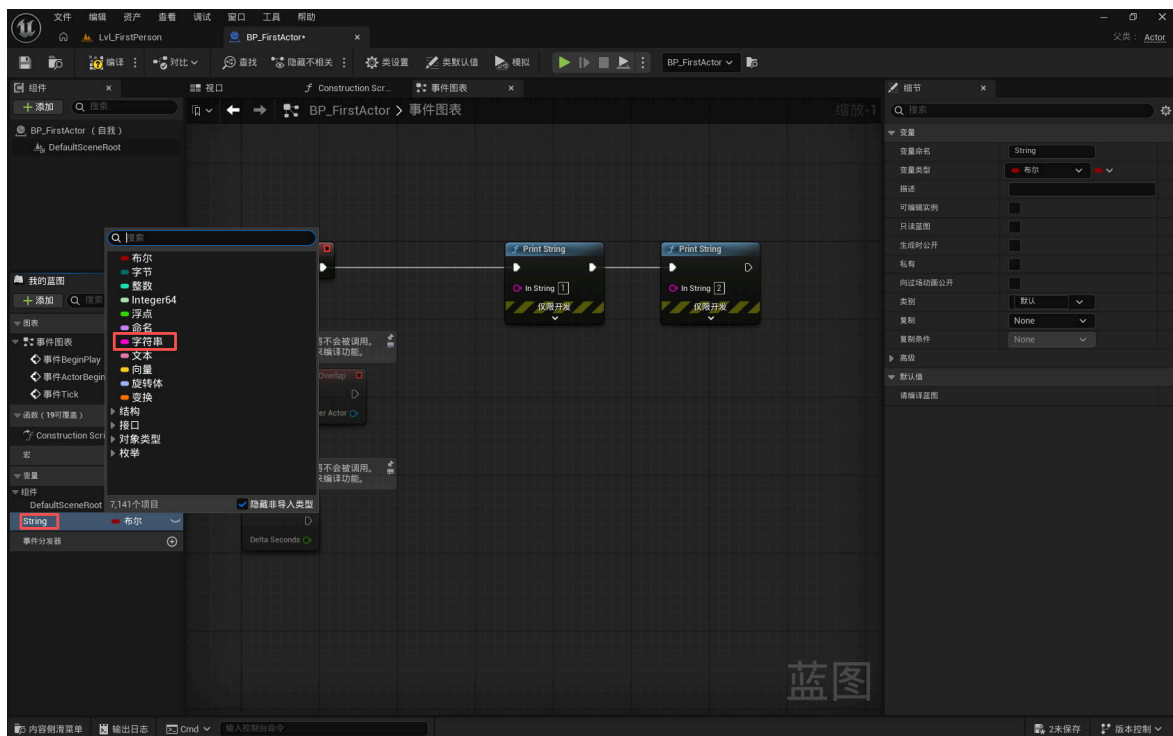


4.5 输入引脚

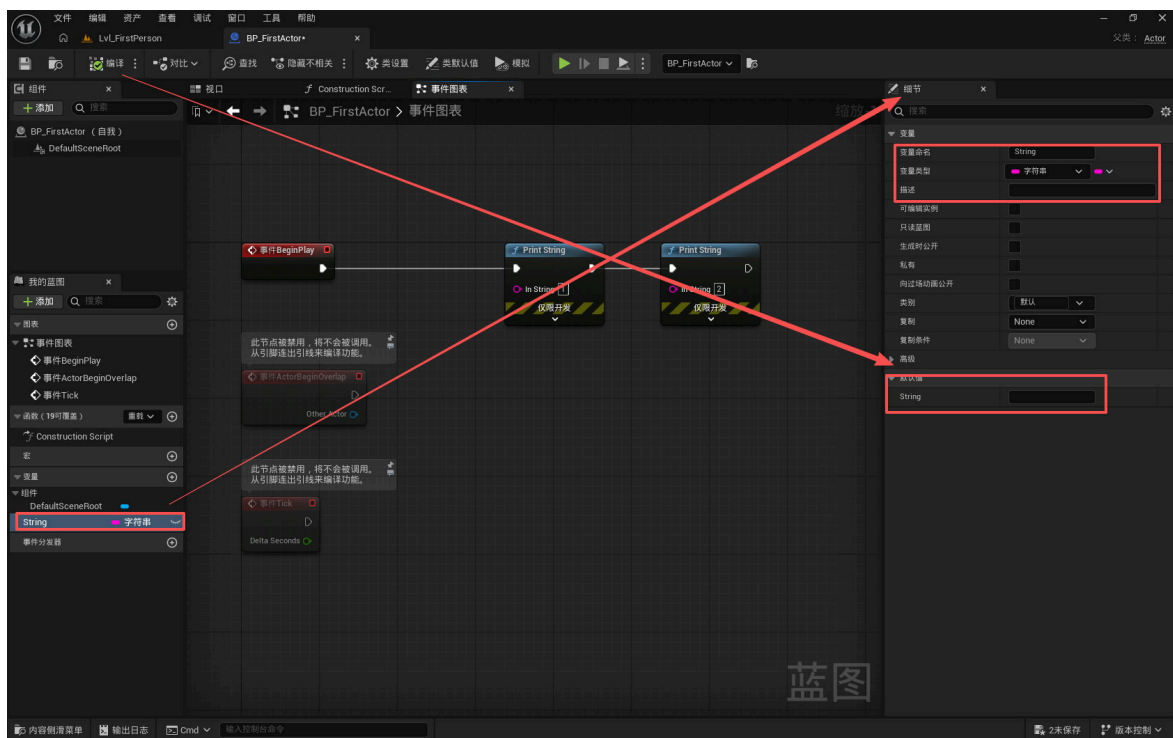
在事件图表界面的左下角变量栏，点击新增变量，修改命名为String



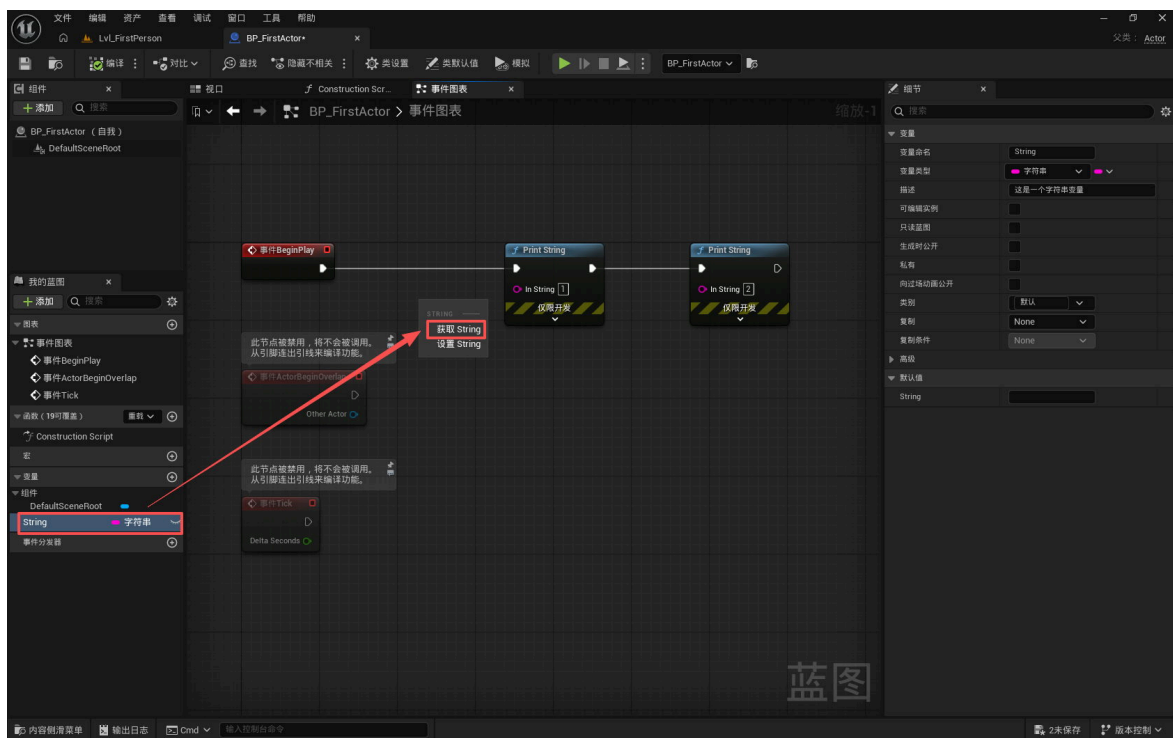
- 如果最开始没有修改命名，可以右键选择重命名或者按F2进行
- 点击变量右侧默认布尔的变量类型，修改变量类型



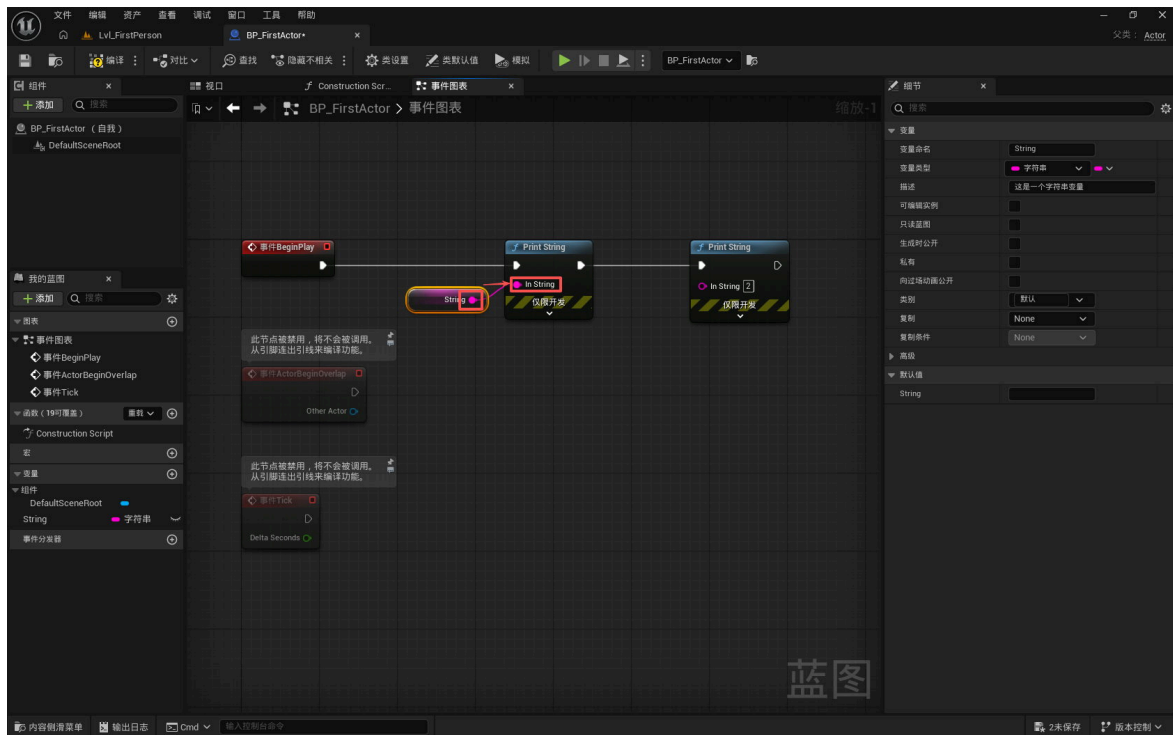
- 左键单击新增的变量，右侧出现该变量细节界面
- 可在细节界面修改变量名、变量类型、变量值，以及添加变量描述
- 修改变量参数需要先完成编译



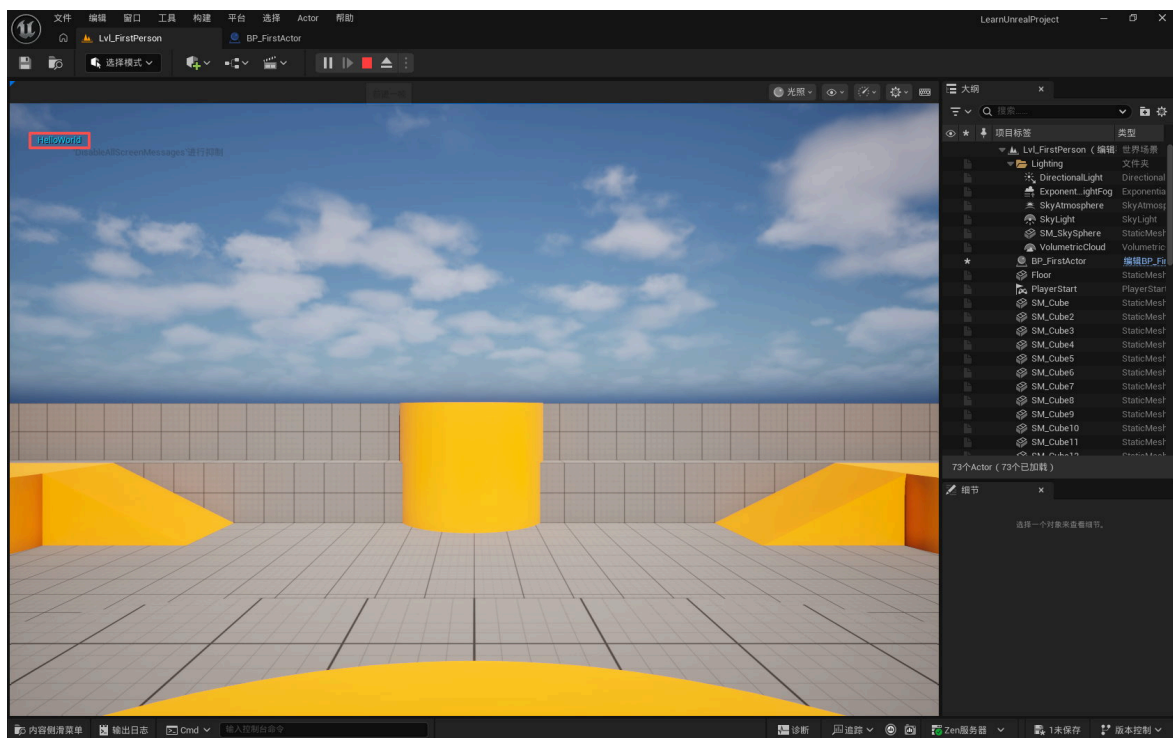
左键长按新增变量，拖拽到事件图表界面内，选择出现的窗口中的获取String



- 点击获取String之后出现新节点，左键长按新节点的右侧，拖拽到需要连接的节点左侧，完成连接
- 连接之后可以观察到，原有的in String后续的可填写内容消失



点击编译后，在关卡内执行运行，效果如图，显示内容为变量值



4.6 取消后续节点操作

删除后续节点：右键节点选择删除，或选中节点后按del键

取消节点连接：先按alt键，再左键节点的执行引脚

